

desktop2web 利用マニュアル

作成 desktop2web project

desktop2web 利用マニュアル

目次

1 技術スタック（確定値）	2
2 設計の柱	2
3 1. skill の配置	4
4 2. hook の登録（必須）	4
5 3. MCP サーバの登録（推奨）	4
6 4. ツール類	4
7 5. 開始	4
8 ⑤ 解析 — 事実の記録に徹する	5
9 ④ 方針決定 — このプロジェクトで一番重い判断	6
10 画面イタレーション — あなたの主戦場	6
10.1 Plan：票を起票する	6
10.2 Do：モックを見て指摘する（承認不要・何度でも）	7
10.3 確定：design-fixed（ここが承認ゲート）	8
10.4 Check / Act：実装と E2E	8
11 機能・DB レーン — legacy-reverse と同じ	8
12 ⑥ 完了検証 — 「機能落ち」を機械で捕まえる	9
13 WBS サイト	10
14 モックのレビュー	10
15 あなたが直接書いてよいファイル	10
16 成果物の全体像	12
17 コマンド早見表	12
18 状態記号	12

C# WinForms のデスクトップアプリを Web アプリへ派生開発するための Claude Code skill 群です。
2つのレーンを並走させます。

表 1

レーン	対象	方針	単位
画面レーン	UI	作り直し（デスクトップ制約から解放して UX を上げる）	1 画面 = 1 イタレーション
機能・DBレーン	ビジネスロジック・データ	鏡移し（挙動は変えない）	1 関数 = 1 イタレーション

全体の流れは次のとおりです。

⑩解析 → ④方針決定 → （画面レーン：モック→レビュー→確定→実装→E2E / 機能レーン：①仕様書→②テスト仕様→③テストコード→④実装→⑤テスト） → ⑥完了検証 → ⑦分析・改善

機能・DB レーンは **legacy-reverse skill** をそのまま流用します（C#→Python の規約だけ差し替え）。そのため機能側の操作は legacy-reverse のマニュアルと同一です。

1 技術スタック（確定値）

表 1-1

層	技術
レガシー	C# WinForms（Designer.cs+イベントハンドラ+ADO.NET/ORM を解析）
バックエンド	Python（機能の鏡移し先）
フロントエンド	React + TypeScript + Tailwind CSS（Vite）
E2E	Playwright（画面ごと）

2 設計の柱

1. **画面は N:M で再編してよい** — 旧画面 1 枚 = 新画面 1 枚に縛られません。「一覧＋詳細モーダル」を「一覧＋サイドパネル」に統合するなど、Web に合う形へ組み替えます。
2. **画面レーンはビジネスロジックを 1 行も書かない** — バリデーション・計算・絞り込み条件はすべて機能レーン（Python）の担当です。フロントで「仮実装」したくなったら、それは⑩の抽出漏れなので機能として追加起票します。この境界を機構で守ることで、UX を自由に変えても業務ロジックは鏡移しのまま保たれます。
3. **試行錯誤は安く、確定だけ重く** — モックは静的 HTML で何版でも作り、承認は不要。design-fixed（デザイン確定）だけが承認ゲートです。指摘は必ず画面票に残ります。
4. **プル型スケジューリング** — 画面が使う機能が⑤pass するまで、その画面の実装には入りません（モック・デザイン確定までは並行可）。WBS に「🕒 機能待ち」と出ます。

あなた（人）の仕事は「トリガを引く」「方針を決める」「モックをレビューして確定させる」「質問に答える／裁定する」です。コードやドキュメントを直接書く必要は基本的にありません。

order-web desktop2web WBS

公開
2026年7月26日

④ 方針書: [approved](#) CRUD: [マトリクス](#)

目次

[進捗サマリ](#)
Open ISSUES（要人間判断）
画面イタレーション
機能一覧（鏡移しレーン）

進捗サマリ

画面 e2e-pass	機能 ⑤pass
0/2	1/3

Open ISSUES（要人間判断）

なし 🚩

画面イタレーション

ID	画面	旧画面	uses(⑤済)	モック	状態	E2E
S-001	注文一覧・詳細	L-001, L-002	1/2 ⌚ 機能待ち	v3	design-fixed	—
S-002	顧客検索	L-003	0/1 ⌚ 機能待ち	v0	draft	—

機能一覧（鏡移しレーン）

関数	①spec	②test-spec	③test-code	④impl	⑤test
F-0001	✓	✓	✓	✓	✓
F-0002	□	□	□	□	□
F-0003	✓	✓	□	□	□

図 2-1: WBS。画面イタレーション（状態・モック版数・E2E）と機能一覧（①～⑤）が 1 画面に集約される。「uses(⑤済) 1/2 ⌚ 機能待ち」は、この画面の実装に入るには機能があと 1 つ必要という意味

セットアップ

3 1. skill の配置

```
cp -r skills/desktop2web      <project>/.claude/skills/  
cp -r skills/legacy-reverse   <project>/.claude/skills/  
cp -r skills/desktop2web/skills/* <project>/.claude/skills/  
cp -r skills/legacy-reverse/skills/* <project>/.claude/skills/
```

機能レーン は legacy-reverse の資産（スクリプト・hook・MCP）をそのまま使うため、**両方を配置する必要があります**。

4 2. hook の登録（必須）

legacy-reverse/hooks/settings-example.json を <project>/.claude/settings.json にマージします。
④⑤中の backend/tests/ 編集を機械的に拒否する安全装置です（E2E freeze 後は frontend/e2e/ も同じ扱いになります）。

5 3. MCP サーバの登録（推奨）

mcp-servers/legacy-reverse-mcp を .mcp.json に登録すると、台帳操作やテスト実行が 型付きツール になり許可プロンプトが減ります。

6 4. ツール類

- Quarto (HTML/PDF) : 未導入なら quarto-typst-pdf skill の qtpdf.py install
- Python: pip install pytest sphinx sphinx-rtd-theme (⑦は radon ruff bandit pip-audit)
- Node: React (Vite) と Playwright (npm i -D @playwright/test && npx playwright install)

7 5. 開始

/desktop2web で状態を確認 → 未セットアップなら /d2w-0-analyze から。

表 7-1

フェーズ	コマンド	人がやること
① 解析	/d2w-0-analyze	対象・DB・パッケージ名を答える。規約を確定。CRUD 検出事項（孤児テーブル等）を裁定
④ 方針	/d2w-policy	概要（2～3 案）を比較して決める 。新画面マップ・UX の原則・API コントラクト
画面	/d2w-screen S-xxx	モックをレビューして確定させる。E2E fail の裁定
機能	/legacy-1-spec ～ /legacy-5-test	①②の承認、⑤fail の裁定（legacy-reverse と同じ）
⑥ 検証	/d2w-6-check	不備リストを見て差し戻し先を判断
⑦ 改善	/legacy-7-analyze	代表ワークロードを教える。施策票を承認

8 ① 解析 — 事実の記録に徹する

AI は画面（Form/UserControl・遷移・スクリーンショット採取）、機能、DB スキーマ、CRUD を洗い出します。このフェーズで**新画面の設計はしません**（それは④）。

人が答えるべきこと：対象ソリューションの場所、DB の種類と DDL の所在、新パッケージ名。

①の重要な副産物が**CRUD マトリクス**です。「どの機能からも触られていないテーブル」「どの画面からも呼ばれない機能」が出てくるので、死蔵か抽出漏れかをあなたが裁定します。

order-web desktop2web WBS ④方針書 **CRUD** 規約

CRUDマトリクス

機能 × テーブル

機能	T_ORDER	T_ORDER_ITEM	T_CUSTOMER	T_AUDIT_LOG
F-0001 search_orders	R		R	
F-0002 save_order	CU	CUD		C
F-0003 find_customer			R	

画面 × 機能

画面	機能
S-001 注文一覧・詳細	F-0001, F-0002
S-002 顧客検索	F-0003

検出事項

- 全テーブルがいずれかの機能から参照されている

目次

機能 × テーブル
画面 × 機能
検出事項

図 8-1: CRUD マトリクス。機能×テーブルの操作（C/R/U/D）と画面×機能の対応が並び、末尾の「検出事項」に孤児テーブルが警告として出る

イベントハンドラ直書きのロジック（`btnSave_Click`の中に計算や検証が書かれている類）は、④で関数として切り出され `origin: ui-embedded` として登録されます。これも鏡移しの対象です。

9 ④ 方針決定 — このプロジェクトで一番重い判断

AI が概案を 2〜3 案の比較で出します。あなたが決めるのは 3 つです。

1. **新画面マップ**（旧画面 N:M 対応） — 以後の WBS の行がここで決まります
2. **UX の原則** — 全モックの判断基準になる「このプロジェクトの憲法」です。例：モーダルの連鎖禁止／検索ボタン式→即時絞り込み／エラーはインライン表示
3. **API コントラクト方針** — 機能をどうエンドポイントに束ねるか

旧画面の 1:1 コピーが既定にならないよう、AI は再編を検討した痕跡を必ず残します（1:1 のままにする場合も「なぜそれが最善か」を書きます）。承認すると方針書が `approved` になり、画面票の骨子が自動生成されます。

10 画面イタレーション — あなたの主戦場

1 画面につき票 1 枚（`docs/screens/S-xxx.md`）で、次の順に進みます。

10.1 Plan：票を起票する

AI が旧画面のスクショと UX 問題点、新画面の狙い、**受け入れ基準**（Given/When/Then）を書きます。受け入れ基準は後で E2E テストの元ネタになるので、ここは目を通してください。なお「計算結果が正しいか」は機能レーン⑤が担保済みなので、受け入れ基準は**画面としての振る舞い**（表示・操作・遷移・エラー表示）に限定されます。

注文一覧・詳細

旧画面の分析

旧画面	役割	UX問題点
L-001 FrmOrderList	条件入力＋検索ボタン＋グリッド	検索ボタンを押すまで結果が出ない。 列幅・並び替えが固定。行を選ぶには 水平スクロールが必要
L-002 FrmOrderDetail	明細の参照・編集（モーダル）	モーダルなので一覧を見ながら編集できない。 閉じると検索条件が消える。 エラーはメッセージボックスで、どの 項目が悪いかわからない

目次

旧画面の分析

新画面の狙い（Plan）

受け入れ基準（design-fixedで凍結）

モック履歴

確定デザイン

実装・E2E記録

新画面の狙い（Plan）

一覧と詳細を1画面に統合し、詳細は**右サイドパネル**で開く（方針書 UX原則1）。検索は入力に応じた即時絞り込み（原則2、デバウンス300ms）。保存時の検証エラーは該当項目の下にインライン表示する（原則3）。行選択はキーボードでも可能にする（原則4）。

受け入れ基準（design-fixedで凍結）

#	Given	When	Then
1	注文が3件ある	画面を開く	3件が注文日の降順で表示される
2	一覧が表示されている	顧客名に “yama” と入力する	300ms 後に該当行のみに絞られ、件数表示が更新される
3	一覧が表示されている	行をクリックする	右サイドパネルに明細が表示され、一覧は絞り込み状態を保つ
4	サイドパネルが開いている	数量を空にして保存する	数量欄の下にインラインでエラーが表示され、保存されない
5	サイドパネルが開いている	Esc を押す	パネルが閉じ、一覧の選択行と絞り込みが保持される

図 10-1: 画面票。旧画面の UX 問題点 → 新画面の狙い → 受け入れ基準（E2E の元ネタ）が 1 枚に並ぶ

10.2 Do：モックを見て指摘する（承認不要・何度でも）

AI が `frontend/mocks/S-xxx/v1.html`（静的 HTML＋Tailwind CDN）を作ってブラウザに出します。あなたは見て指摘するだけです。指摘は**必ず票のモック履歴表に転記**されるので、口頭のやり取りが消えません。

表 10-1

v	レビュー指摘（要約）	対応
1	詳細をモーダルで出していた（原則 1 違反）。列が旧画面のまま 12 列あって読めない	サイドパネル化。列を 6 列に絞る
2	検索が「絞り込む」ボタン式のまま。件数が分からない	即時絞り込み＋件数表示を追加
3	指摘なし。これで確定	—

注文管理 / 注文一覧

mock v3

顧客名

yama

状態

すべて

1件 / 全3件

注文ID

顧客名 ▼

状態

注文日

金額

明細

1001

Yamada Shoji

受注

2026-01-05

128,000

3

絞り込み条件に一致しない行は非表示 (TANAKA Corp / Suzuki KK)

注文 #1001

Yamada Shoji

注文日

2026-01-05

数量

数量は必須です (1以上の整数)

単価

42,000

金額

128,000

保存

キャンセル

検証・計算はバックエンド (F-0002 save_order) が担当。フロントは表示と入力のみ。

図 10-2: モック v3 (確定版)。旧画面のモーダル+検索ボタン式を、サイドパネル+即時絞り込み+ インラインエラーに置き換えている。右下の注記どおり、検証・計算はバックエンドの担当

10.3 確定 : design-fixed (ここが承認ゲート)

「これで確定」と言うと、票が **design-fixed** になり**受け入れ基準が凍結**されます。以後にデザインを変えなくなったら ISSUE → 承認 → モックレビューに差し戻し、が正規経路です (版数は続きから)。

10.4 Check / Act : 実装と E2E

AI が React 実装 → Playwright E2E (受け入れ基準 1 行=1 テスト) を作って freeze → 実行します。fail 時の裁定は機能レーン⑤と同型です。

表 10-2

分類	意味	あなたの操作
(a) 画面実装のバグ	受け入れ基準を満たしていない	何もしない (AI が直して再実行)
(b) E2E が基準とズレ	テスト側の誤り	ISSUE を見て承認 → E2E 再生成
(c) 受け入れ基準・機能仕様が誤り	上流の誤り	ISSUE で裁定 → 票または①へ差し戻し

3 回失敗すると自動で ISSUE が起票され、WBS に ➊ が出てあなたの裁定待ちになります。

11 機能・DB レーン — legacy-reverse と同じ

①仕様書のレビュー、②テスト仕様の承認、⑤failの裁定 ((a)実装／(b)テストコード／(c)仕様) まで、操作は legacy-reverse マニュアルのとおりです。C# 特有の点だけ補足します。

- `decimal` は `Decimal` に (金額を `float` にしない)、`DataTable` は `list[dict]` か `dataclass`、`out/ref` 引数は戻り値タプルに——という型対応表を⑥で確定します
- `origin: ui-embedded` の機能は「UI から引き剥がしたロジック」です。UI への参照を引数化したシグネチャになっているか、①のレビューで特に注意して見てください

order-web desktop2web WBS ④方針書 CRUD 規約							Q
テスト結果: search_orders							目次
サマリ							サマリ
							ケース別結果
							失敗詳細
							試行履歴
仕様ケース数	実装済み	実装率	成功	失敗	スキップ	実行時間	
2	2	100%	2	0	0	0.00s	
ケース別結果							
ケースID	内容	分類	実装	結果	時間	詳細	
F0001-TC-001	顧客名未指定は全件・注文日降順	正常系	✓	✓	0.0s		
F0001-TC-002	顧客名は大文字小文字を無視した部分一致	正常系 / 境界値	✓	✓	0.0s		
失敗詳細							
なし							
試行履歴							
試行	日時	実装率	結果				
1	2026-07-26T21:18	100%	pass				

図 11-1: テスト結果報告書。実装率（②の全ケースが③に実装されたか）とケースごとの内容・分類・結果が 1 枚で分かる

12 ⑥ 完了検証 — 「機能落ち」を機械で捕まえる

/d2w-6-check は次を検証します。

- 1. 方針書が approved／全画面が e2e-pass（E2E が改変されていないことも照合）
- 2. 全機能が⑤pass（legacy-reverse の⑥基準）
- 3. **CRUD 突合** — 旧側にあった 機能×テーブル 操作が新側に全部あるか（消えた操作＝機能落ち）、旧側になかった操作が増えていないか（増えた操作＝勝手な機能追加）を両方向でチェック
- 4. **旧画面カバレッジ** — 全ての旧画面がいずれかの新画面票に現れているか（拾い忘れ検出）
- 5. open ISSUE ゼロ

pass すると最終レンダリング（HTML サイト＋PDF＋Sphinx API）まで進みます。

13 WBS サイト

```
python -m http.server 8765 --directory docs/_site
```

トップが WBS です。上から順に：方針書の状態、進捗サマリ、**Open ISSUES（あなたへの質問）**、画面イタレーション表、機能一覧。ナビバーから方針書・CRUD・規約・⑥完了検証へ行けます。

14 モックのレビュー

```
python -m http.server 8767 --directory frontend/mocks
```

モックは静的 HTML なのでビルド不要で開けます。AI に「S-001 のモックを見せて」と言えばブラウザに出します。

15 あなたが直接書いてよいファイル

表 15-1

ファイル	手編集	説明
docs/conventions.md • docs/domain-knowledge.md	○	あなたが著者
ISSUE の「回答（人が記入）」欄	○	じっくり書きたい裁定はファイルで
docs/policy.md	○	方針の追記・修正（画面票の判断基準が変わる点に注意）
docs/screens/S-xxx.md の受け入れ基準	△	design-fixed 前なら自由。後は ISSUE 経由
docs/specs/（機能仕様書）	△	編集可。ハッシュ連鎖で②③が stale になり再確認が走る
WBS・CRUD・テスト結果・完了検証	✕	自動生成。再生成で消える

ファイルに直接書いた内容は、次にどのフェーズを起動したときに AI が自動で拾います。

表 15-2

症状	意味	対処
WBS に「🕒 機能待ち」	その画面が使う機能が⑤未達	機能レーンを先に回す（ <code>/legacy-1-spec</code> から）。モック確定までは並行可
WBS に 🛑 ISSUE-xxx	ループが上限到達、裁定待ち	ISSUE に回答 → unblock → 再トリガ
機能一覧が stale⚠	①が改訂され②が古い	<code>/legacy-2-testspec</code> で再生成→再承認
CRUD に孤児テーブル警告	どの機能からも参照されないテーブル	死蔵か抽出漏れかを裁定 (ISSUE)
モックにロジックが書かれている	画面レーンの越境	機能として追加起票させる (<code>functions.json</code> + ISSUE)
<code>frontend/e2e/</code> の編集が拒否された	freeze 後の保護が正常動作	ISSUE→承認→再生成が正規経路

16 成果物の全体像

```
docs/
  index.qmd          WBS (自動生成・手編集禁止)
  policy.md          ④方針書 (承認ゲート)
  crud.md            CRUD マトリクス (自動生成)
  screens/S-xxx.md    画面票 (draft→mock-review→design-fixed→implemented→e2e-pass)
  specs/ test-specs/ test-results/ issues/ 機能レーン (legacy-reverse と同一)
  conventions.md     domain-knowledge.md completion-check.md
data/
  screens.json        functions.json  schema.json  ①の解析結果 (正データ)
  ledger.json         機能レーンの台帳
backend/src/ , backend/tests/ ④実装 / ③テスト (hook で保護)
frontend/src/         React 実装
frontend/mocks/S-xxx/v<N>.html モック (試行錯誤用)
frontend/e2e/S-xxx.spec.ts Playwright (design-fixed 後に freeze)
```

17 コマンド早見表

```
python <d2w>/scripts/d2w_ledger.py wbs      # 画面+機能の統合 WBS
python <d2w>/scripts/d2w_ledger.py crud     # CRUD マトリクス
python <d2w>/scripts/d2w_ledger.py next     # 次にやる画面と待ち機能
python <d2w>/scripts/d2w_ledger.py screen-skeletons # 画面票の骨子生成
python <lr>/scripts/ledger.py verify <func-id> # 機能レーンのハッシュ連鎖検証
```

<d2w> = desktop2web skill のルート、<lr> = legacy-reverse skill のルート。

18 状態記号

☑=完了 / □=未着手 / ⌚=前提待ち / ❶=裁定待ち / ⚠=要再確認 / —=対象外・未実行